

HPC Important Questions – Easy English Answers (Unit 1 to Unit 6)

Unit 1: Introduction to Parallel Computing

Parallel computing improves speed, handles large problems, uses resources efficiently, reduces cost, and saves energy.

Moore's Law increased transistor count, leading to multi-core processors and parallel programming. Memory performance is limited by latency, bandwidth, cache coherence, power use, and the memory wall problem.

SIMD runs one instruction on many data, MIMD runs different instructions on different data, SPMD runs same program on different data, and SMT runs threads together.

Flynn's classification includes SISD, SIMD, MISD, and MIMD.

Parallel platforms use shared memory, distributed memory, hybrid memory, and network-based systems.

Communication cost increases execution time and reduces speedup and scalability.

Modern processors use multi-core design, SMT, GPUs, SIMD extensions, and heterogeneous computing.

Pipelining overlaps instruction stages to improve instruction-level parallelism.

Unit 2: Parallel Algorithm Design

Parallel algorithms use decomposition, task scheduling, communication control, synchronization, and scalability.

Decomposition techniques include recursive, data, exploratory, speculative, and hybrid methods.

A task dependency graph shows task order and execution dependencies.

Granularity defines task size and affects efficiency and load balance.

The critical path is the longest dependent task sequence and limits performance.

Static and dynamic mapping assign tasks to balance load.

Communication overhead is reduced using locality, aggregation, and overlap.

Unit 3: Message Passing (MPI)

Message passing uses explicit send and receive between processes with private memory.

Blocking communication waits for completion, while non-blocking continues execution.

A communicator defines a group of communicating processes.

Collective communication includes broadcast, gather, scatter, allgather, and reduce.

Deadlocks occur due to circular wait or mismatch and are avoided by ordering and non-blocking calls.

MPI_Send and MPI_Recv handle data transfer between processes.

Unit 4: Synchronization and Threads

Barriers synchronize all processes, locks ensure mutual exclusion, and semaphores control access.

Dependencies include flow, anti, output, and control dependencies.

Pthreads create threads using pthread_create and synchronize using mutex and condition variables.

OpenMP uses compiler directives for parallel regions, loops, critical sections, and reduction.

False sharing occurs when different variables share a cache line and reduces performance.

Unit 5: GPGPU and CUDA

GPGPU uses GPUs for general-purpose parallel computation.

GPUs have many small cores, while CPUs have fewer powerful cores.

CUDA kernels run in parallel on GPUs.

GPU memory types include global, shared, local, and constant memory.

CUDA uses thread, block, and grid hierarchy for scalability.

Unit 6: Performance Measures

Amdahl's Law shows speedup is limited by the serial portion of a program.

Gustafson's Law allows problem size to grow and gives better speedup for large problems.